

ロボコンの無線通信と帯域共有の実測評価

他チーム通信・ROS 2・Bluetooth・ESP-NOW

貝淵蒼馬

要旨 指令と大容量データを同時に流し、他チームの通信、チャンネル幅、QoS、指令周期を比較した。実測 SDR 画像は帯域の重なりを示す。一方、指令が期限内に届くかはアプリの応答記録から評価する。チャンネルを分離しても端末内部や同じチーム内の遅延は残り、帯域幅・平均速度・接続表示だけでは操縦品質を判断できなかった。

1 目的と実験方法

スマートフォンからの指令、カメラ映像、LiDAR 点群を同じ Wi-Fi と ROS 2 で運ぶロボットを想定する。自チーム内だけでなく、隣のチームも大容量通信中の条件を作った。観測 PC (Intel BE200) が Ch6・20 MHz AP、ラップトップ 2 (MT7921E) がロボット役、ラップトップ 1 (Intel AX210) が有線でインターネットを共有する。ESP32 3 台は指令、別チーム AP / STA、独立 BLE、ESP-NOW へ順次使用した。

先行試験は 64 byte・100 Hz UDP 指令と TCP 模擬データを STA → AP → STA へ通した。追加の実 ROS 2 試験は PC AP → STA (指令側 Humble・ロボット側 Jazzy、双方 CycloneDDS)、Image と PointCloud2 を各 10 Hz で生成する。合成センサの要求 payload は 60.54 Mbps、実受信は約 47–55 Mbps だった。実カメラ、スマートフォン、モータは使用していない。経路の異なる系列を混ぜて方式の順位を決めない。

1 試行 30 秒・3 反復を基本とし、ESP-NOW 長時間系列だけ 600 秒・2–3 反復とする。RTT p99 は応答が戻った指令の分位点、20 ms 期限超過率は全予定指令が分母で欠落も含む。最新 sequence の更新途絶も評価する。20 ms は比較目標で、ロボットの安全限界ではない。環境の反復数はパケット数とは区別する。

2 他チーム通信とチャンネル配置

表 1: 自チーム大容量 TCP 中。値は他チーム待機／負荷。各 3 反復。

他 Ch	RTT p99 [ms]	20 ms 超過 [%]
6	125.8 / 148.4	55.0 / 67.7
7	123.8 / 132.3	53.8 / 63.0
11	126.5 / 127.4	54.4 / 57.2

他チームを負荷へ変えると、p99 の増分は同一 Ch6 で約 23 ms、隣接 Ch7 で約 9 ms、分離 Ch11 で約 1 ms だった。別のチャンネル番号でも帯域が分かれるとは限らない。Ch6 と Ch7 は中心が 5 MHz しか離れず、20 MHz 幅で大きく重なる。同一 Ch が今回最も遅く、隣接が常に最悪という結果ではない。

同一帯域の送信待ち、部分重複による受信妨害、再送などが候補になる [1]。再送数やキャリアセンスを直接測っておらず、色だけで原因を特定しない。他チーム実受信量も約 0.4–4.7 Mbps と異なるため、等しい干渉電力や空中時間の比較ではない。

2.1 チャンネル幅と周波数の余裕

Ch1 の 20 MHz は名目上 2402–2422 MHz、Ch6 は 2427–2447 MHz。Ch1 + 副 Ch5 の 40 MHz では約 2402–2442 MHz となり Ch6 へ重なる。20 → 40 MHz で p99 は 128.6 → 127.5 ms、期限超過は 55.0 → 56.8% だった。帯域が広いほど同時配置の余裕は減るが、転送が速くなれば同じデータの送信時間は短くなる。広いほど必ず悪いとは言えず、実受信量と指令期限を併せて判断する。今回の幅比較は他チーム AP の介入で、自チーム AP の最高速度比較ではない。

2.2 基準性能の確認

操縦のみ・他待機でも UDP の p99 は 117–120 ms、20 ms 超過は 47–49% だった。既に目標を満たさない構成なので、全遅延を他チームへ帰属しない。RTT を 2 で割って片道値にすることや、別 PC 時計の差を未同期で使うことはできない。

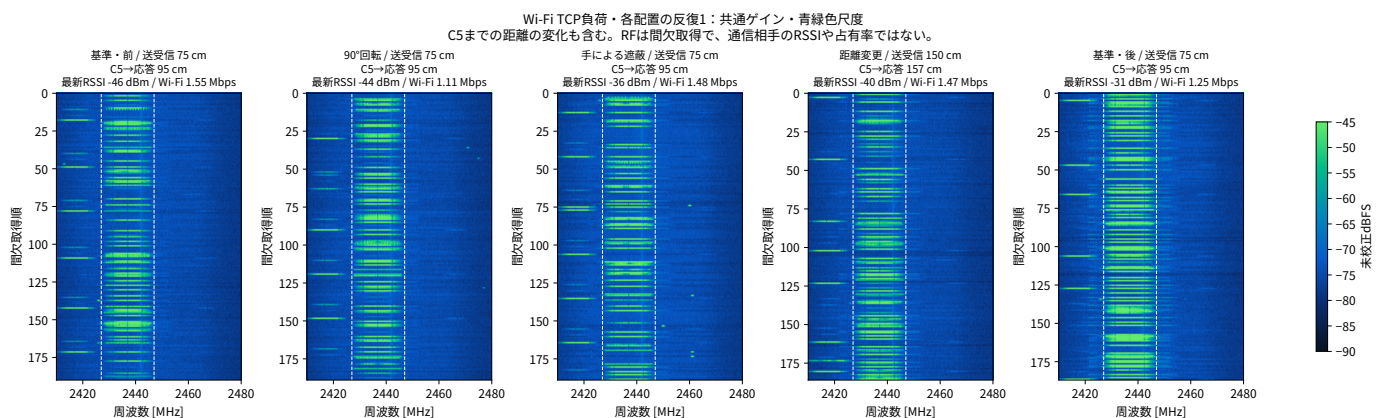
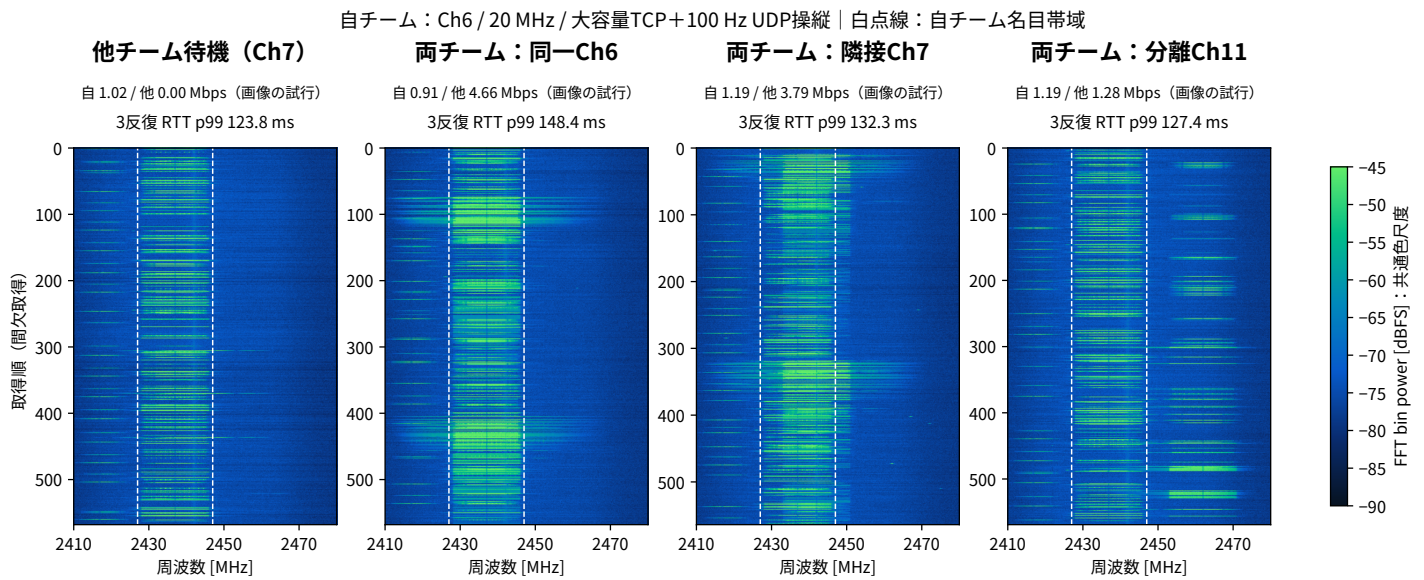


図 1: 短い配置比較の Wi-Fi 負荷・反復 1。中心間距離と C5 までの距離を併記。共通ゲイン・未校正 dBFS。



3 流量・生成周期・ROS 2 の比較

追加の流量試験では他チームを負荷中に固定し、自チーム TCP の要求量を 0–2 Mbps へ変えた。同一 Ch6 の 0 → 1 Mbps で 20 ms 超過は 54.08 → 72.61%、実受信平均は 1.007 Mbps だった。2 Mbps 要求でも実受信は 1.105 Mbps にとどまり、生成 byte が未送信で積み上がった。設定 Mbps を達成速度や無線負荷とみなさず、生成・送信受付・実受信を分ける。

同じ 1 Mbps 要求で生成周期 100 → 10 ms へ変えると、Ch6 の超過は 63.47 → 59.42% だったが、Ch7 では 53.88 → 53.88% だった。小分け生成は常に改善する結果ではなく、TCP や OS は byte をまとめ直せる。平均量に加え時間的な集中と端末の待ち行列を評価する必要がある。

3.1 QoS と大容量トピック

指令 QoS は reliable・深さ 10 に固定し、センサだけ best effort / reliable、深さ 1 / 10 を比較した。指令のみの 20 ms 超過は約 45%、センサ追加時は約 70–80% となった。best effort・深さ 1 でも同一 Ch6 の p99 は 145.57 ms、超過 77.68% だった。深さ 1 は DDS の履歴設定であり、無線待ち行列を 1 つにしたり指令を優先したりする設定ではない [2]。画像の unique sequence 初回受信間隔には最大 3.33 秒の途絶もあった。受信間隔は観測できたが、生成から到着までのデータの古さは未同期時計から計算していない。

3.2 省電力と遅延の切り分け

端末の省電力 ON / OFF を実際に切り替え、前後の readback を確認した。指令のみでは両方 p99 約 103–109 ms が残った。一方センサ負荷・他 Ch11 では OFF → ON で 163.38 → 248.66

ms だった。省電力 OFF だけで今回の遅延は解消しない。予定送信のずれは約 0.5–1 ms、publish 呼出し約 0.08 ms、ロボット内の応答前処理約 0.003–0.007 ms だった。publish が速くても、その後の DDS・OS・MAC の待ちは残る。p99 を引き算して原因の寄与を割り当てない。

4 SDR 画像の読み方

C5 : 80 MS/s、16,380 I/Q、アナログ帯域 48 MHz、ゲイン 20、FFT1024。各行約 205 μ s、名目 RF 観測時間比約 0.4% であり、縦軸は間欠取得順。青緑は未校正 dBFS で、同じ図の共通尺度内で比較する。緑の割合を占有率、通信成功率、絶対 dBm へ換算しない。Ch11 の一部は名目アナログ帯域の端へかかり、弱い色を無干渉の証拠としない。画像で帯域の重複、アプリ記録で届く時刻、実受信量で負荷の実現度を確認する。

4.1 ESP-NOW の固定チャンネルと Wi-Fi の共有

ESP-NOW は標準で BLE のように自動ホッピングしない。Wi-Fi の無線部から専用 action frame を送り、送受信機は同じチャンネルに合わせる。設定の channel=0 は現在の Wi-Fi チャンネルを使う意味で、自動探索ではない [3]。先行試験の Ch6・7・11 は試行間で変更し、各試行中は固定した。長時間試験と配置比較は Ch6 固定である。

AP を通らず指令経路を分けても、同一・重複帯域では Wi-Fi と送信時間を共有する。別無線なら分離チャンネルを選べるが、同じ ESP32 で AP 接続も併用する場合は AP のチャンネルに合わせる必要がある [5]。指令の回数を増やすとヘッダや応答を含む送信機会も増えるため、payload の Mbps だけで影響を判断しない。

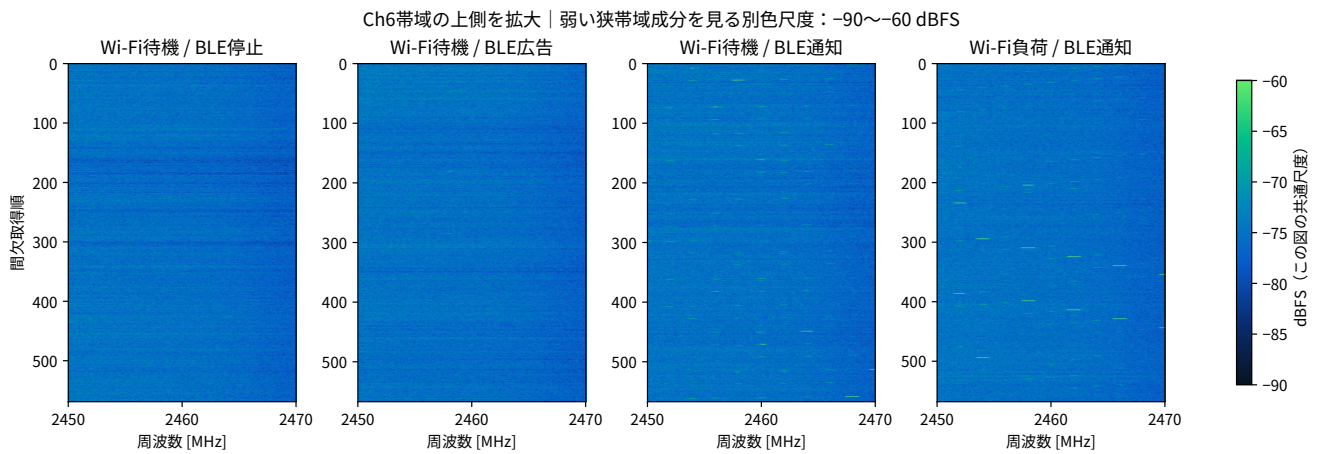


図 3: BLE 停止・広告・GATT 通知。2450–2470 MHz の拡大、反復 1。この図は - 90 ～ - 60 dBFS で前ページとは尺度が異なる。

5 Bluetooth LE との共存

5.1 広告と接続後の周波数の使い方

今回の広告は 2402・2426・2480 MHz の 3 周波数を使う方式である。接続後の GATT 通知は、2 MHz 間隔の 37 データチャンネルから、接続イベントごとに周波数を選ぶ。ホッピングは広帯域を一度に送ることではなく、狭い信号の送信先を時間で切り替えることである。AFH は使用チャンネル集合を更新して混雑した周波数を避ける仕組みだが、今回その更新は測っていない [4]。

Wi-Fi Ch6 の名目 20 MHz 帯域 2427–2447 MHz 内にも BLE データチャンネルがある。同じ周波数へ同時に送れば干渉し得るため、ホッピングしても Wi-Fi と無関係にはならない。40 MHz 化は重なり得る周波数を増やすが、実送信時間も変わるので劣化量は幅だけで決まらない。

5.2 双方の実測性能

独立した ESP32 2 台で BLE 停止・20 ms 設定の広告・BLE 1M 通知を各 3 反復比較した。Wi-Fi 大容量通信中の p99 は停止 126.2 / 広告 126.0 / 通知 126.3 ms と近いが、20 ms 超過は 54.6 / 58.2 / 56.8% だった。Wi-Fi 実受信平均は 1.34 / 1.50 / 1.19 Mbps、BLE 通知は Wi-Fi 待機 0.365 → 負荷 0.326 Mbps だった。p99 が同じでも、期限超過や双方の配送量まで同じではない。少数反復・実負荷の違いがあり、これを衝突や AFH の効果だけに帰属しない。図の細線は狭帯域成分の手掛かりで、全てを BLE と同定したものではない。間欠取得から連続ホッピングを復元せず、同一チップの共存制御や Bluetooth 音声も未検証である。

6 ESP-NOW 指令の周期と更新途絶

6.1 今回確認した影響と指令周期

2 台間で 64 byte を往復し、20・50・100・200 Hz、別系統 Wi-Fi Ch6 の待機 / TCP 負荷を各 600 秒測った。使用者の終了希望で 19 試行 (8 条件、各 2–3 反復) とし、残り 5 試行は未実施。960,000 予定指令を全て送信し、959,997 unique 応答を記録した。MAC 送信成功はアプリ応答やアクチュエータ動作成功とは異なる [3]。

表 2: 長時間系列の Wi-Fi 負荷条件。欠落も超過に含む。

Hz	p99 [ms]	20 ms 超過 [%]	最大更新途絶 [ms]
20	9.18	0.0167	100.7
50	9.62	0.0267	46.5
100	9.26	0.0008	23.9
200	14.26	0.2108	36.8

100 Hz・負荷中は p99 9.26 ms だったが、200 Hz では 14.26 ms、10 秒窓の超過が集中する区間もあった。指令周期を短くすれば必ず通信が良くなるわけではない。20 Hz の通常更新間隔自体が 50 ms であるため、1 応答の 20 ms 期限と継続更新の周期は別に設計する。200 Hz の payload は片道 0.1024 Mbps だが、ヘッダ・ACK・送信待ちも必要で空中時間とは一致しない。先行 30 秒試験の Ch6 は Wi-Fi 待機→負荷で p99 が 9.4 → 85.6 ms だったが、負荷中の反復差も大きい。実装の異なる長時間系列へ混ぜない。Wi-Fi の実受信は約 1.7–2.7 Mbps だった。ただし ESP-NOW 停止時の同一条件 Wi-Fi 基準がなく、ESP-NOW が Wi-Fi 速度を何 % 下げたかは測定していない。主に Wi-Fi 負荷中に ESP-NOW 指令が間に合うかを評価した。AP 中継 UDP / ROS 2 とは無線経路も異なり、方式だけの勝敗は決められない。

6.2 短時間の配置比較

送受信 75 cm、C5 → 送信 45 cm、C5 → 応答 95 cm を基準に、応答機の 90° 回転、手による遮蔽、送受信 150 cm への距離変更、基準復帰を各 10 秒・3 反復で比較した。負荷中 p99 は基準前 10.91 / 回転 7.79 / 遮蔽 7.93 / 距離変更 8.83 / 基準後 8.63 ms で、全配置の負荷中 20 ms 超過は 0%。全 30 試行・30,000 指令で応答欠落はなかった。基準復帰後も p99 が下がったため、回転や遮蔽による改善とは断定できない。届いた最新パケットの RSSI だけで欠落時の感度は判断できない。C5 までの距離、ケーブルや反射を含め、短時間の結果を到達距離の保証へ広げない。

7 ロボコン運用への示唆

本番前にカメラ・点群を動かし、隣のチームにも通信を流してもらい、単独基準と同条件で比較する。チャンネル分離、生成量、周期、QoS、省電力は一つずつ変え、平均速度と期限超過・欠落・最長更新途絶を残す。古い指令を追いつくまで送るより、最新指令の扱いと途絶時の動作を設計する。後者は今回モータで検証しておらず、実機の停止期限は別に確かめる必要がある。

参考文献

- [1] Cisco, Wireless RF Reference Guide.
- [2] ROS 2 Jazzy, Quality of Service settings.
- [3] Espressif, ESP-NOW Programming Guide.
- [4] Bluetooth SIG, Bluetooth LE Primer.
- [5] Espressif, ESP-NOW FAQ.